SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – CODING CHALLENGE QUESTIONS

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO1 – Precision (BL4)	**Assessment:**	Coding challenge questions
Weightage:	40%	**Total Marks:**	100
CO1 Statement:	Analyse the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking		
Student Name:		**Reg. No.:**	
Section / Batch:		**Date:**	

Instructions to Students

- Analyze the given scenario and identify the problem requirements before developing the solution.
- Design the algorithm or flowchart and select the appropriate 8085 instructions, registers, and addressing modes required for implementation.
- Develop and execute the 8085 Assembly Language Program (ALP) using a simulator or trainer kit to solve the given problem.
- Verify and validate the output by testing the program with the given input data and ensuring the expected results are obtained.
- Interpret the program execution by explaining the logic used, the role of registers, memory locations, flags, and the final output generated.

QUESTIONS

1. A warehouse stores the quantity of products available in eight storage racks. Before dispatch, the warehouse manager wants the processor to arrange the quantities in ascending order and identify the rack with the lowest stock for replenishment. Write an 8085 ALP to
    - Sort the quantities in ascending order.
    - Store the sorted values.
    - Identify and store the minimum quantity.
2. During data transmission, a block of eight bytes is copied from one memory region to another. To ensure successful transmission, each byte in the destination memory must be compared with its corresponding source byte. Write an 8085 ALP to
    - Copy the data block.
    - Compare each source byte with the copied byte.
    - Count the number of mismatches.
    - Store the mismatch count.
3. A communication controller receives an 8-bit binary code. According to the communication protocol, the code is valid only if it contains an even number of 1s. Develop an 8085 ALP to
    - Count the number of set bits.
    - Determine whether the parity is even or odd.
    - Store a validity flag in memory.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

GNUSim8085 - 8085 Microprocessor Simulator

File Reset Assembler Debug Help

Registers

A	00	
BC	00	00
DE	81	08
HL	80	08
PSW	00	00
PC	42	1E
SP	FF	FF
Int-Reg	00	

Flag

S 0
Z 1
AC 0
P 1
C 0

Decimal - Hex Convertion

Decimal: 0 — To Hex
Hex: 0 — To Dec

I/O Ports

0 − + 00
Update Port Value

Memory

0 − + 00
Update Memory

Load me at

```
1          LXI H,8000H
2          LXI D,8100H
3          MVI C,08H
4
5    LOOP: MOV A,M
6          XCHG
7          MOV M,A
8          XCHG
9
10         CMP M
11         JZ NEXT
12
13         INR B
14
15   NEXT: INX H
16         INX D
17         DCR C
18         JNZ LOOP
19
20         MOV A,B
21         STA 8200H
22
23         HLT
```

Data | Stack | KeyPad | **Memory** | I/O Ports

Start 8200h OK

Address (Hex)	Address	Data
8200	33280	0
8201	33281	0
8202	33282	0
8203	33283	0
8204	33284	0
8205	33285	0
8206	33286	0
8207	33287	0
8208	33288	0
8209	33289	0
820A	33290	0
820B	33291	0

Line No	Assembler Message
0	Program assembled successfully

Simulator: Idle

GNUSim8085 - 8085 Microprocessor Simulator

File Reset Assembler Debug Help

Registers

A	00	
BC	00	00
DE	00	00
HL	80	00
PSW	00	00
PC	42	28
SP	FF	FF
Int-Reg	00	

Flag

S 0
Z 1
AC 0
P 1
C 1

Decimal - Hex Convertion

Decimal: 0 — To Hex
Hex: 0 — To Dec

I/O Ports

0 − + 00
Update Port Value

Memory

0 − + 00
Update Memory

Load me at

```
1          LXI H,8000H
2          MVI C,07H
3
4   OUTER: LXI H,8000H
5          MVI B,07H
6
7   INNER: MOV A,M
8          INX H
9          CMP M
10         JC NOSWAP
11         JZ NOSWAP
12
13         MOV D,M
14         MOV M,A
15         DCX H
16         MOV M,D
17         INX H
18
19  NOSWAP: DCR B
20         JNZ INNER
21
22         DCR C
23         JNZ OUTER
24
25         LXI H,8000H
26         MOV A,M
27         STA 8100H
28
29         HLT
```

Data | Stack | KeyPad | **Memory** | I/O Ports

Start 8000h OK

Address (Hex)	Address	Data
8000	32768	0
8001	32769	0
8002	32770	0
8003	32771	0
8004	32772	0
8005	32773	0
8006	32774	0
8007	32775	25
8008	32776	0
8009	32777	0
800A	32778	0
800B	32779	0

Line No	Assembler Message
0	Program assembled successfully

Simulator: Idle

GNUSim8085 - 8085 Microprocessor Simulator

File Reset Assembler Debug Help

Registers

A	01	
BC	00	04
DE	81	08
HL	80	08
PSW	00	00
PC	42	27
SP	FF	FF
Int-Reg	00	

Flag

S 0
Z 1
AC 1
P 1
C 0

Decimal - Hex Convertion

Decimal: 0 — To Hex
Hex: 0 — To Dec

I/O Ports

0 − + 00
Update Port Value

Memory

0 − + 00
Update Memory

Load me at

```
1   LDA 8000H
2   MVI B,08H
3   MVI C,00H
4
5   LOOP: RAR
6   JNC SKIP
7   INR C
8
9   SKIP: DCR B
10  JNZ LOOP
11
12  MOV A,C
13  STA 8100H
14
15  ANI 01H
16  JZ EVEN
17
18  MVI A,00H
19  STA 8101H
20  JMP STOP
21
22  EVEN: MVI A,01H
23  STA 8101H
24
25  STOP: HLT
```

Data | Stack | KeyPad | **Memory** | I/O Ports

Start 8000h OK

Address (Hex)	Address	Data
8000	32768	90
8001	32769	0
8002	32770	0
8003	32771	0
8004	32772	0
8005	32773	0
8006	32774	0
8007	32775	25
8008	32776	0
8009	32777	0
800A	32778	0
800B	32779	0

Line No	Assembler Message
0	Program assembled successfully

Simulator: Idle